

Comprehensive Internship Project Report: Iris Flower Classification

Author / Intern: Data Science Intern

Internship Program: Oasis Infobyte Data Science Internship

Task Track: Task 1 Iris Flower Classification

Domain: Machine Learning, Pattern Recognition & Multiclass Classification

Associated Notebook: Iris_Classification.ipynb

Date: September 2026

Executive Summary

During my data science internship, my first assigned project was to develop an end-to-end machine learning classification system capable of identifying the species of an Iris flower (`Setosa`, `Versicolor`, or `Virginica`) from four physical measurements: Sepal Length, Sepal Width, Petal Length, and Petal Width.

While the Iris flower dataset (introduced by British statistician and biologist Ronald Fisher in 1936) is often considered the "Hello World" of pattern recognition, I approached this task not merely as a basic coding exercise, but as a complete, professional machine learning project.

I systematically executed all phases of the data science lifecycle:

1. Data Ingestion & Schema Verification
2. Data Hygiene Audit & Missing Value Verification
3. Exploratory Data Analysis (EDA) & Multivariate Visualizations
4. Feature Discrimination Analysis via ANOVA F-Statistics
5. Data Preprocessing & Stratified Train-Test Splitting
6. Feature Standardization Pipeline Implementation
7. Multi-Algorithm Training (Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest)
8. Hyperparameter & Cross-Validation Evaluation
9. Confusion Matrix & Classification Metric Analysis
10. Model Serialization & Production Inference Testing

The resulting classifier achieved **100% Accuracy** on the independent test set and **96.67% ($\pm 2.11\%$)** on 5-Fold Stratified Cross-Validation. After comparing all models, **Logistic Regression** was declared the best-performing model due to its high generalization stability, low variance, and Occam's razor simplicity.

1. Problem Statement & Objectives

1.1 Biological & Data Context

The Iris dataset contains 150 instances of Iris flowers, evenly divided across three species (50 instances per species):

- **Iris-setosa**
- **Iris-versicolor**
- **Iris-virginica**

Each sample consists of four continuous numerical features measured in centimeters (cm):

11. **Sepal Length (cm)**: The length of the outer protective leaf of the flower.
12. **Sepal Width (cm)**: The width of the outer protective leaf of the flower.
13. **Petal Length (cm)**: The length of the inner decorative petal of the flower.
14. **Petal Width (cm)**: The width of the inner decorative petal of the flower.

1.2 Project Goals

As an intern, my primary objectives were:

- **Build a Robust Data Pipeline**: Ensure proper data loading, non-null verification, and clean pandas DataFrame structuring.
- **Perform Comprehensive EDA**: Use Seaborn and Matplotlib to visualize feature distributions, identify feature correlations, and discover underlying class separability.
- **Conduct Quantitative Feature Importance Analysis**: Compute statistical metrics (such as ANOVA F-scores) to prove which measurements are most discriminative.
- **Implement Proper Machine Learning Practices**: Prevent data leakage by separating train and test datasets before feature scaling, and preserve class balances using stratified splitting.
- **Compare Diverse Classifiers**: Train linear, distance-based, tree-based, and ensemble algorithms under identical experimental conditions.
- **Select and Justify the Best Model**: Evaluate models beyond basic accuracy by analyzing confusion matrices, precision, recall, F1-scores, and cross-validation stability.

2. Environment Setup & Tooling

To ensure full reproducibility, I configured a standardized Python environment utilizing core scientific and machine learning libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import joblib

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import f_classif

# Machine Learning Classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Evaluation Metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

warnings.filterwarnings('ignore')

# Global Visualization Configuration
%matplotlib inline
sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
```

3. Step-by-Step Execution Workflow

Step 3.1: Ingestion & DataFrame Structuring

I began by loading the raw dataset directly from `sklearn.datasets.load_iris()`. The raw dataset comes as a dictionary-like object with separate arrays for features (`data`) and numerical class indices (`target`).

To make the data human-readable and convenient for analysis, I constructed a structured pandas DataFrame, explicitly assigning column headers and mapping the numeric targets (0, 1, 2) to their corresponding species names:

```
raw_iris = load_iris()
df = pd.DataFrame(data=raw_iris.data, columns=raw_iris.feature_names)
df['target'] = raw_iris.target
df['species'] = df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})
```

Dataset Dimensions: 150 rows × 6 columns.

Step 3.2: Exploratory Data Analysis (EDA) & Data Hygiene Audit

Before building any models, I conducted a rigorous exploratory data audit:

- 15. **Missing Value Audit:** Ran `df.isnull().sum()` to inspect for missing or NaN values.
 - Result: **0 missing values** across all columns. The dataset is complete.
- 16. **Data Types Check:** Inspected column data types via `df.info()`.
 - All four feature columns (`sepal length`, `sepal width`, `petal length`, `petal width`) are stored as continuous `float64` numerical values.
- 17. **Class Balance Verification:** Checked sample counts per target species using `df['species'].value_counts()`.
 - Setosa: 50 samples (33.3%)
 - Versicolor: 50 samples (33.3%)
 - Virginica: 50 samples (33.3%)
 - Result: The dataset is perfectly balanced, meaning standard classification accuracy is a valid evaluation metric.
- 18. **Descriptive Summary Statistics:** Computed summary statistics (Mean, Std, Min, 25%, Median, 75%, Max) overall and grouped per species.

Summary Statistics Table (Overall)

Feature Name	Mean (cm)	Std Dev (cm)	Min (cm)	25% IQR (cm)	Median (50%)	75% IQR (cm)	Max (cm)
Sepal Length	5.843	0.828	4.300	5.100	5.800	6.400	7.900

Sepal Width	3.057	0.436	2.000	2.800	3.000	3.300	4.400
Petal Length	3.758	1.765	1.000	1.600	4.350	5.100	6.900
Petal Width	1.199	0.762	0.100	0.300	1.300	1.800	2.500

Summary Statistics Grouped by Species (Mean ± Std Dev)

Species Name	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)
Setosa	\$5.006 \pm 0.352\$	\$3.428 \pm 0.379\$	\$1.462 \pm 0.174\$	\$0.246 \pm 0.105\$
Versicolor	\$5.936 \pm 0.516\$	\$2.770 \pm 0.314\$	\$4.260 \pm 0.470\$	\$1.326 \pm 0.198\$
Virginica	\$6.588 \pm 0.636\$	\$2.974 \pm 0.322\$	\$5.552 \pm 0.552\$	\$2.026 \pm 0.275\$

Step 3.3: Data Visualizations & Pattern Recognition

To gain visual intuition into feature relationships, I generated three key visualization figures:

1. Pairwise Feature Distribution Matrix (Pairplot)

Using Seaborn's `sns.pairplot`, I rendered a 4×4 scatter matrix with Kernel Density Estimate (KDE) curves along the diagonal, color-coded by species (``Setosa`` = teal, ``Versicolor`` = orange, ``Virginica`` = green).

Key Visual Takeaways:

- ``Setosa`` forms a completely isolated, linearly separable cluster in every plot involving petal measurements.
- ``Versicolor`` and ``Virginica`` show slight overlap along sepal measurements, but remain distinct along petal dimensions.
- Petal length and petal width show a clear positive linear relationship across all species.

2. Feature Distribution Box Plots & Swarm Plots

I created a 2×2 subplot grid showing box plots combined with strip plots (swarm points) for each measurement grouped by species.

Key Visual Takeaways:

- **Petal Length:** `Setosa` ranges between 1.0 cm and 1.9 cm, `Versicolor` ranges between 3.0 cm and 5.1 cm, and `Virginica` ranges between 4.5 cm and 6.9 cm. The interquartile ranges (IQRs) do not overlap at all between `Setosa` and the other two species.
- **Petal Width:** Similarly displays clear separation: `Setosa` (0.1 - 0.6 cm), `Versicolor` (1.0 - 1.8 cm), `Virginica` (1.4 - 2.5 cm).
- **Sepal Width:** `Setosa` has the highest mean sepal width (3.43 cm), whereas `Versicolor` has the lowest (2.77 cm). However, there is significant overlap between `Versicolor` and `Virginica`.

3. Pearson Correlation Heatmap

I computed the Pearson correlation matrix for all four continuous features and plotted an annotated heatmap.

Correlation Matrix Values:

- `Petal Length` $\rightarrow$ `Petal Width`: **+0.963** (Extremely strong positive correlation)
- `Sepal Length` $\rightarrow$ `Petal Length`: **+0.872** (Strong positive correlation)
- `Sepal Length` $\rightarrow$ `Petal Width`: **+0.818** (Strong positive correlation)
- `Sepal Length` $\rightarrow$ `Sepal Width`: **-0.118** (Very weak negative correlation)

Step 3.4: Quantitative Feature Selection Analysis

To validate my visual findings statistically, I performed an Analysis of Variance (**ANOVA F-test**) using `sklearn.feature_selection.f_classif`. The ANOVA F-statistic measures the ratio of between-class variance to within-class variance. A higher F-statistic indicates greater feature discriminative power.

```
X_raw = df[raw_iris.feature_names]
y_raw = df['target']
f_values, p_values = f_classif(X_raw, y_raw)
```

ANOVA F-Statistic Feature Ranking Table

Rank	Feature Name	ANOVA F-Statistic Score	p-Value	Discriminative Assessment
1	Petal Length (cm)	1180.16	$< 10^{-15}$	Extremely High Separability

2	Petal Width (cm)	960.01	$< 10^{-15}$	Extremely High Separability
3	Sepal Length (cm)	119.26	$< 10^{-15}$	Moderate Separability
4	Sepal Width (cm)	49.16	$< 10^{-15}$	Lowest Separability (Overlapping)

Conclusion: Petal length and petal width are the primary drivers for separating Iris species.

Step 3.5: Preprocessing, Stratified Split & Feature Scaling

1. Train-Test Split Ratio

I selected an **80/20 Stratified Split**:

- **Training Set:** 120 samples (40 Setosa, 40 Versicolor, 40 Virginica)
- **Test Set:** 30 samples (10 Setosa, 10 Versicolor, 10 Virginica)

I specified `stratify=y` inside `train_test_split` with `random_state=42` to guarantee that both training and testing datasets maintained exact 1:1:1 class balance.

2. Feature Standardization Pipeline

Distance-based models (like KNN) and gradient-based models (like Logistic Regression) are sensitive to feature scales. I applied `StandardScaler` to transform features to zero mean ($\mu = 0$) and unit variance ($\sigma = 1$):

$$z = \frac{x - \mu}{\sigma}$$

Preventing Data Leakage:

I explicitly fit the scaler strictly on the training set and then transformed both training and test sets:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 3.6: Machine Learning Classifier Comparison

I selected four algorithms representing different machine learning paradigms:

19. Logistic Regression (Linear Model):

- Configuration: `max_iter=200`, `random_state=42`, fitted on scaled features.

- Paradigm: Estimates log-odds probabilities using linear decision boundaries.

20. K-Nearest Neighbors (KNN, Instance-Based Model):

- Configuration: `n\_neighbors=5`, Euclidean distance metric, fitted on scaled features.
- Paradigm: Classifies a query sample based on majority vote of its 5 nearest neighbors.

21. Decision Tree Classifier (Rule-Based Non-Linear Model):

- Configuration: `max\_depth=3`, Gini impurity criterion, `random\_state=42`.
- Paradigm: Hierarchical decision rules splitting feature space into orthogonal hyper-rectangles.

22. Random Forest Classifier (Ensemble Bagging Model):

- Configuration: `n\_estimators=100`, `random\_state=42`.
- Paradigm: Ensemble of 100 decision trees trained on bootstrap sub-samples.

4. Experimental Evaluation & Results Summary

Each model was evaluated across multiple metrics:

- **Training Accuracy:** To check for underfitting/overfitting.
- **Test Accuracy:** Proportion of correct predictions on unseen test data.
- **5-Fold Stratified Cross-Validation:** Mean and standard deviation across 5 resampled folds.
- **Macro Precision, Recall & F1-Score:** Unweighted mean of metrics across all 3 classes.
- **Confusion Matrix:** Visualized as a 2D Seaborn heatmap.

Comprehensive Model Performance Table

Classifier Model	Train Accuracy	Test Accuracy	5-Fold CV Mean Acc	5-Fold CV Std Dev	Macro Precision	Macro Recall	Macro F1-Score
Logistic Regression	1.0000	1.0000	0.9667	±0.0211	1.0000	1.0000	1.0000
K-Nearest Neighbors (\$K=5\$)	0.9750	1.0000	0.9667	±0.0211	1.0000	1.0000	1.0000
Decision Tree (<code>max_depth=3`</code>)	0.9833	0.9667	0.9600	±0.0327	0.9697	0.9667	0.9665
Random Forest	1.0000	0.9667	0.9600	±0.0327	0.9697	0.9667	0.9665

(`n\_estimators=100`)

Detailed Confusion Matrix Breakdown (Test Set, \$N=30\$)

1. Logistic Regression & KNN Confusion Matrix

	Predicted Setosa	Predicted Versicolor	Predicted Virginica
True Setosa	10	0	0
True Versicolor	0	10	0
True Virginica	0	0	10

- **Result:** Perfect Classification! \$30 / 30\$ test samples correctly identified (\$0\$ errors).

2. Decision Tree & Random Forest Confusion Matrix

	Predicted Setosa	Predicted Versicolor	Predicted Virginica
True Setosa	10	0	0
True Versicolor	0	10	0
True Virginica	0	1	9

- **Result:** 1 misclassification (\$29 / 30\$ correct). 1 `Virginica` sample was misclassified as `Versicolor`.

5. Best Model Declaration & Technical Rationale

Declared Best Model: Logistic Regression (with `StandardScaler`)

Technical Rationale:

23. **Top Generalization Performance:** Reached **100% Test Accuracy** and tied for top **5-Fold Cross-Validation Accuracy (96.67%)**.
24. **Lowest Variance across Folds:** Exhibited the lowest cross-validation standard deviation ($\sigma = \pm 0.0211$), proving consistent stability across data splits.
25. **Occam's Razor & Model Simplicity:** On smaller dataset sizes (\$N=150\$), simpler linear models are far less prone to variance or overfitting than complex non-linear decision trees or ensemble models.
26. **Linear Separability Alignment:** EDA proved that `Setosa` is linearly separable and `Versicolor`/`Virginica` are cleanly demarcated in standardized feature space, making a linear log-odds decision hyperplane mathematically optimal.

6. Model Serialization & Live Sample Inference

I saved the trained best model and scaler pipeline to disk using `joblib`:

```
joblib.dump(best_model, 'best_iris_model.pkl')
joblib.dump(scaler, 'iris_scaler.pkl')
```

Live Inference Test Results:

I tested predictions on three custom flower measurement samples:

Sample #	Sepal Length	Sepal Width	Petal Length	Petal Width	Model Prediction	Probability / Confidence	Ground Truth Match
Sample 1	\$5.1 ex t{ cm}\$	\$3.5 ex t{ cm}\$	\$1.4 ex t{ cm}\$	\$0.2 ex t{ cm}\$	SETOSA	99.81%	Setosa
Sample 2	\$6.2 ex t{ cm}\$	\$2.9 ex t{ cm}\$	\$4.3 ex t{ cm}\$	\$1.3 ex t{ cm}\$	VERSICOLOR	95.42%	Versicolor
Sample 3	\$7.3 ex t{ cm}\$	\$2.9 ex t{ cm}\$	\$6.3 ex t{ cm}\$	\$1.8 ex t{ cm}\$	VIRGINICA	98.15%	Virginica

7. Personal Reflections & Internship Key Learnings

- 27. **End-to-End Execution Habits:** I learned the importance of structuring a data science workflow systematically from data hygiene checks to cross-validated model selection rather than jumping straight to fitting a classifier.
- 28. **Data Leakage Awareness:** I gained a practical understanding of why preprocessing transformers (StandardScaler) must be fit strictly on training data, preserving test data as completely unseen evaluation data.
- 29. **Statistical Feature Ranking:** Using ANOVA F-tests reinforced how statistical hypothesis testing complements visual data exploration.

30. **Model Selection Philosophy:** I discovered that the most complex model (Random Forest) is not always the best choice. For clean, low-dimensional, linearly separable data, simpler models like Logistic Regression are more reliable, interpretable, and resilient.